

The Guard Personal Safety AI

A comprehensive technical blueprint for building an AI-powered personal safety application that combines real-time computer vision with on-device large language model reasoning to protect users in any situation.

Hybrid Mobile Architecture
Flutter + YOLOv8 + Llama.cpp + LangChain
Privacy-First Design

Table of Contents

1. Introduction and Vision	3
2. System Architecture	3
2.1 Architecture Overview	4
3. Phase 1: The Setup (Foundations)	4
3.1 Flutter Project Initialization	5
3.2 Choosing the AI Brain Host	5
Option A: On-Device LLM (High End)	5
Option B: Local Server (Stable)	6
4. Phase 2: The Eyes (Computer Vision)	6
4.1 Model Selection and Preparation	6
4.2 Flutter Integration	7
4.3 Fall Detection Logic	7
5. Phase 3: The Agent (AI Reasoning)	8
5.1 Local LLM Setup	8
5.2 The Guard Persona (System Prompt)	9
5.3 The Reasoning Loop	9
6. Phase 4: Action and Connectivity	10
6.1 Voice Interface (The Check-In)	10
6.2 Emergency Contacts and SMS Notification	10
7. Phase 5: Privacy and Guard Mode	11

7.1 Zero Cloud Upload Architecture	11
7.2 Black Box Mode	12
8. Key GitHub Repositories	12
9. Getting Started: First Steps	13
9.1 Step-by-Step First Milestone	13
9.2 Development Timeline	13

1. Introduction and Vision

The Guard is a personal safety AI application designed to serve as a constant, intelligent watchdog over the user or their loved ones. By combining cutting-edge computer vision, on-device language model reasoning, and seamless emergency connectivity, The Guard represents the frontier of AI engineering applied to personal security. The application runs entirely on the user's device or a local edge server, ensuring that no sensitive video or audio data is ever transmitted to the cloud. This privacy-first architecture is central to the design philosophy, making The Guard one of the most trustworthy safety companions available.

The name "The Guard" was chosen because it evokes a sense of strength, protection, and reliability. Much like a dedicated security professional, the application watches over the user in real time, analyzing visual and auditory inputs to detect potentially dangerous situations. Whether it is an elderly family member who may have suffered a fall, a solo traveler walking home at night, or anyone who wants an extra layer of personal safety, The Guard is designed to be their intelligent, ever-vigilant companion. The application does not merely react to predefined triggers; it uses a local large language model (LLM) to reason about situations contextually, distinguishing between genuine emergencies and benign events with nuance and accuracy.

Building an application that combines real-time computer vision with on-device LLM reasoning is a challenging but immensely rewarding engineering endeavor. This roadmap outlines the complete technical development journey, from initial setup through production deployment, covering every major system component, architectural decision, and implementation phase. The document is organized into five development phases, each building upon the previous one, along with dedicated sections on privacy, resource management, and the key open-source repositories that power the system.

Vision Statement: Build an AI-powered personal safety application that runs entirely on-device, combining real-time object detection with intelligent reasoning to protect users and their loved ones in any situation, without ever compromising their privacy.

2. System Architecture

The Guard employs a Hybrid Mobile Architecture that optimizes the balance between processing speed, battery efficiency, and user experience. The architecture is designed around three core principles: locality (all sensitive processing happens on-device or on a local edge server), modularity (each major subsystem operates independently and can be upgraded or replaced without

affecting others), and responsiveness (the user interface remains smooth and responsive even during intensive AI processing). This hybrid approach ensures that the application performs reliably on a wide range of devices, from flagship smartphones to modest home servers.

2.1 Architecture Overview

The system is composed of four distinct layers, each responsible for a critical aspect of the application's functionality. These layers communicate through well-defined interfaces, ensuring that changes to one layer do not cascade into others. The following table summarizes each layer, its technology stack, and its role within the overall system.

Layer	Technology	Role
Frontend (Interface)	Flutter (Dart)	Cross-platform UI for iOS and Android with high-performance rendering and native-feel interactions
Computer Vision (Eyes)	YOLOv8 (TFLite/ONNX)	Real-time object detection running directly on the device for low-latency visual analysis
AI Brain (Agent)	Llama.cpp (Phi-3 Mini)	Local LLM reasoning engine for contextual decision-making and intelligent response generation
Backend (Orchestration)	Python + LangChain	Connects vision and language subsystems, manages data flow, and coordinates actions

Table 1: System Architecture Layers

The frontend layer, built with Flutter, provides a unified codebase that compiles to native iOS and Android applications. Flutter was selected for its excellent cross-platform performance, rich widget library, and strong community support. The computer vision layer uses YOLOv8, specifically the Nano variant, converted to TensorFlow Lite or ONNX format for efficient on-device inference. The AI reasoning layer leverages llama.cpp to run quantized large language models such as Microsoft's Phi-3 Mini or Meta's Llama-3-8B, enabling sophisticated contextual reasoning without any cloud dependency. Finally, the orchestration layer, written in Python using LangChain-inspired logic, coordinates the flow of data between vision and language, triggering appropriate actions based on the AI agent's decisions.

3. Phase 1: The Setup (Foundations)

Before any AI logic can be implemented, the development environment must be properly configured to handle real-time video processing and on-device inference. This foundational phase establishes the project structure, configures platform-specific permissions, and sets up the AI compute infrastructure. Getting this phase right is critical, as it determines the performance ceiling for every subsequent development stage. A poorly configured environment can introduce latency, memory leaks, and platform incompatibilities that become increasingly difficult to resolve as the application grows in complexity.

3.1 Flutter Project Initialization

The first step is to create a new Flutter project using the standard Flutter CLI tool. The project should be initialized with the latest stable version of Flutter to ensure compatibility with the most recent packages and APIs. After creating the project skeleton, the developer must configure platform-specific camera access permissions. On Android, this involves adding the appropriate uses-permission declarations to the AndroidManifest.xml file, including permissions for the camera, microphone, and foreground services (for background monitoring). On iOS, the Info.plist must include the NSCameraUsageDescription and NSMicrophoneUsageDescription keys with clear, user-facing descriptions of why the app needs these permissions.

The Flutter project structure should be organized with clear separation of concerns from the outset. Recommended directory structure includes separate folders for the vision module, the AI agent module, the communication module (voice and SMS), and shared utilities. This modular approach makes the codebase more maintainable and allows different team members to work on different subsystems simultaneously without creating merge conflicts.

3.2 Choosing the AI Brain Host

One of the most important architectural decisions is where the LLM runs. There are two viable options, each with distinct trade-offs that must be carefully evaluated based on the target hardware, battery constraints, and performance requirements.

Option A: On-Device LLM (High End)

For flagship smartphones equipped with a neural processing unit (NPU), the LLM can run directly on the phone using llama.dart, which is a Dart-native port of the popular llama.cpp C++ library. This approach eliminates all network latency and ensures the AI functions even without internet connectivity. However, it demands significant RAM (typically 4-8 GB for a quantized model) and can rapidly drain the device battery during sustained inference. Modern devices with Snapdragon 8 Gen 3 or Apple A17 Pro chips are well-suited for this configuration, as their NPUs can handle the

matrix operations required by transformer models with remarkable efficiency.

Option B: Local Server (Stable)

For broader device compatibility and better battery life, the LLM can run on a small home server such as a Raspberry Pi 5, an Intel NUC, or even a laptop that remains on the same Wi-Fi network as the phone. The phone connects to this server via a local HTTP API, sending context data and receiving AI-generated decisions. This approach, powered by LocalAI (an open-source drop-in replacement for OpenAI's API that runs entirely locally), provides a stable and power-efficient alternative. The main trade-off is the introduction of network latency (typically 50-200ms on a local network), which is generally acceptable for safety-critical reasoning but must be accounted for in the system's timeout and failover logic.

Criteria	Option A: On-Device	Option B: Local Server
Latency	Minimal (0-10ms)	Low (50-200ms)
Battery Impact	High drain	Minimal
Hardware Required	Flagship phone with NPU	Raspberry Pi / NUC / Laptop
Offline Support	Full	Requires Wi-Fi
Recommended For	High-end deployments	General use, home setup

Table 2: AI Brain Host Options Comparison

4. Phase 2: The Eyes (Computer Vision)

The computer vision subsystem is responsible for transforming raw camera frames into structured data that the AI agent can reason about. This phase covers the entire visual pipeline, from model selection and conversion to real-time integration with the Flutter camera and the implementation of the critical fall detection logic. The vision system must operate with minimal latency, as safety-critical applications demand real-time or near-real-time analysis of the user's environment. A delay of even a few seconds could mean the difference between a timely alert and a missed emergency.

4.1 Model Selection and Preparation

The Ultralytics YOLOv8 family provides a range of models optimized for different trade-offs between speed and accuracy. For The Guard, the Nano variant (yolov8n.pt) is the recommended starting point. At approximately 6.3 million parameters, it achieves inference speeds of over 100 frames per second on modern smartphone CPUs while maintaining sufficient accuracy for person detection and basic scene understanding. For devices with dedicated NPUs, the Small variant (yolov8s.pt) can be considered for improved accuracy at the cost of higher computational demand.

The downloaded PyTorch model (.pt file) must be converted to a mobile-friendly format. The recommended pipeline involves exporting the model to TensorFlow Lite (.tflite) format using Ultralytics' built-in export function. This conversion optimizes the model's computational graph for mobile inference, applying operator fusion, quantization (INT8 where supported), and layout optimization. The resulting .tflite file is typically 5-15 MB, making it practical for inclusion in the application bundle without excessive download sizes.

4.2 Flutter Integration

Integrating YOLOv8 with the Flutter camera requires the tflite_flutter package, which provides Dart bindings for the TensorFlow Lite runtime. The application opens a camera stream using the official camera package, processes each frame through the TFLite interpreter, and overlays bounding boxes on a custom painter widget. The detectObjects function is the core of this module, accepting a CameraImage and returning a list of detected objects with their class labels, confidence scores, and bounding box coordinates. The function must be optimized to run asynchronously, ensuring that the camera preview remains smooth at 30+ FPS while the detection runs on a separate isolate or thread.

4.3 Fall Detection Logic

Standard YOLOv8 detects objects such as "person" with high reliability, but it does not natively detect falls. The fall detection capability is implemented as a logic layer built on top of the person tracking output. The system tracks the bounding box of each detected person across consecutive frames, analyzing changes in the box's geometry to infer the person's posture and movement.

The core algorithm monitors the ratio of the bounding box's height to its width over time. Under normal conditions (a standing or walking person), the height significantly exceeds the width. When a person falls, the vertical dimension of the bounding box decreases rapidly while the horizontal dimension either stays constant or increases, as the person transitions from an upright to a horizontal position. The system triggers a "Potential Fall Detected" alert when the height-to-width ratio drops below a calibrated threshold (typically 0.6-0.8) within a short time window (typically 1-2 seconds). To minimize false positives, the system also considers factors such as the velocity of

the ratio change and the person's proximity to furniture or other objects that might indicate sitting down rather than falling.

Parameter	Value	Description
Height/Width Ratio Threshold	0.6 - 0.8	Triggers fall alert when ratio drops below this value
Time Window	1 - 2 seconds	Duration within which the ratio change must occur
Confidence Threshold	0.85	Minimum YOLO detection confidence for person class
False Positive Filter	Context-aware	Checks proximity to furniture and rate of change

Table 3: Fall Detection Parameters

5. Phase 3: The Agent (AI Reasoning)

The Agent subsystem is what transforms The Guard from a simple motion detector into an intelligent safety companion. While traditional safety apps rely on rigid if-else logic and fixed threshold triggers, The Guard uses a local large language model to reason about detected events contextually, much like a human safety monitor would. The LLM evaluates the totality of available information, including the nature of the detected event, the user's recent activity patterns, environmental context, and responses to voice check-ins, to make nuanced decisions about the appropriate level of response.

5.1 Local LLM Setup

The recommended models for The Guard's reasoning engine are small but capable language models that can run efficiently on consumer hardware. Microsoft's Phi-3 Mini (3.8 billion parameters) and Google's Gemma 2B (2 billion parameters) are both excellent candidates, offering strong reasoning capabilities in compact form factors. When quantized to 4-bit precision using GGUF format, these models require only 2-4 GB of RAM, making them practical for on-device deployment on modern smartphones or lightweight edge servers. The models are deployed using llama.cpp, a highly optimized C++ inference engine that supports quantized models across multiple hardware platforms including ARM (mobile), x86 (desktop/server), and Apple Silicon.

5.2 The Guard Persona (System Prompt)

The LLM's behavior is shaped by a carefully crafted system prompt that defines its role, capabilities, and constraints. This prompt is injected at the beginning of every inference request, ensuring consistent behavior across all interactions. The Guard persona is designed to be protective, cautious, and decisive, with a clear escalation framework that maps situations to specific actions.

System Prompt: "You are 'The Guard,' a personal safety AI. Your goal is to protect the user. You receive sensory data (e.g., 'Fall detected,' 'User unresponsive'). Based on the severity, decide on an action: 1. Ignore (False alarm), 2. Alert User (Vibration), 3. Emergency Protocol (Contact authorities/family). Respond ONLY with the action number and a short explanation."

The three-tier action framework provides a clean interface between the AI's reasoning and the application's response mechanisms. Action 1 (Ignore) is selected when the AI determines that a detected event is a false alarm, such as the user sitting down quickly or a pet triggering the motion sensor. Action 2 (Alert User) is the most common response, used when the AI detects a potentially concerning event but cannot confirm it is an emergency; this triggers a subtle vibration pattern and a voice check-in asking the user to confirm they are safe. Action 3 (Emergency Protocol) is reserved for high-confidence emergency scenarios, such as a confirmed fall followed by non-responsiveness, which triggers immediate contact with emergency services and designated family members.

5.3 The Reasoning Loop

The reasoning loop is the core operational cycle of The Guard. It begins with sensory input from the vision subsystem, flows through the LLM for contextual reasoning, and produces an action decision that is executed by the connectivity subsystem. The loop runs continuously during active monitoring mode, processing each new frame of sensory data as it arrives.

A typical scenario illustrates the loop in action. The vision subsystem detects a sudden change in the user's bounding box geometry consistent with a fall. This triggers a voice check-in: "Are you okay?" The system waits for a response, typically setting a timeout of 15-20 seconds. If the user responds with a designated safe phrase (such as "I'm fine"), the LLM receives the context: "Vision detected a fall. User responded with safe phrase within 10 seconds." The LLM reasons: "The fall was likely minor. The user's prompt response indicates awareness and mobility. Action 1 is appropriate." If the user does not respond within the timeout, the LLM receives: "Vision detected a fall. Voice check: User did not answer within 15 seconds." The LLM reasons: "The fall combined with non-response indicates high danger. Action 3 is required." This produces the output: "Action

3: Emergency Protocol. Reasoning: High probability of injury."

6. Phase 4: Action and Connectivity

Once The Guard's AI agent has made a decision, the application must translate that decision into concrete actions that protect the user. This phase implements the voice interface for check-ins, the emergency contact notification system, and the GPS location sharing mechanism. The connectivity subsystem must be robust and reliable, as it represents the final link in the safety chain. A failure in this subsystem, even after the AI has correctly identified an emergency, renders the entire system ineffective.

6.1 Voice Interface (The Check-In)

The voice interface serves a dual purpose: it allows The Guard to check on the user when a potentially dangerous event is detected, and it provides the user with a hands-free way to cancel false alarms. The implementation uses the `speech_to_text` package in Flutter for real-time speech recognition and the `flutter_tts` package for text-to-speech output. When a fall or other concerning event is detected, the application uses the text-to-speech engine to ask: "Are you okay? Please say 'I'm fine' if you do not need help." The speech-to-text engine listens for a designated safe phrase or any spoken response. If the user says the safe phrase, the alert is cancelled. If the user says something else or does not respond within the configured timeout (default 20 seconds), the emergency protocol is activated.

The voice interface is designed to work even in noisy environments, using noise-cancellation preprocessing and multiple recognition attempts before declaring non-responsiveness. The system also accounts for users who may be disoriented or in pain after a fall, implementing a simple tap-to-confirm fallback that appears on screen alongside the voice prompt. This multi-modal approach ensures that the system does not escalate to emergency protocols due to a temporary inability to speak.

6.2 Emergency Contacts and SMS Notification

The emergency notification system uses the `url_launcher` and `sms` packages to send pre-composed alert messages to designated emergency contacts. When the AI agent triggers Action 3 (Emergency Protocol), the application constructs an SMS message using the LLM's reasoning output, including the GPS coordinates of the user's last known location, the nature of the detected event, and the user's response status. The message template follows a clear, actionable format:

Emergency SMS Template: "The Guard AI detected a fall at [GPS Location]. User is unresponsive. Emergency services are being notified. Please respond immediately."

The application maintains a list of emergency contacts configured by the user during initial setup, with each contact assigned a priority level. Priority 1 contacts (typically immediate family members) are notified first, followed by Priority 2 contacts (close friends or neighbors) if no Priority 1 contact acknowledges the alert within a configurable time window. The system also provides an option to directly call emergency services (911 or the local equivalent) using the tel: URL scheme, bypassing the SMS system for the most urgent situations.

Action Level	Trigger	Response
Action 1: Ignore	False alarm detected by AI	No action; continue monitoring
Action 2: Alert User	Potential concern detected	Vibration + voice check-in ("Are you okay?")
Action 3: Emergency	Fall + non-response confirmed	SMS to contacts + GPS sharing + call emergency services

Table 4: Three-Tier Action Framework

7. Phase 5: Privacy and Guard Mode

Privacy is not an afterthought in The Guard; it is a foundational design principle that influences every architectural decision. The application is designed with a strict "zero cloud upload" policy for video and audio data. All sensitive processing happens in the device's memory (RAM) and is never written to persistent storage or transmitted over the network, except in the specific case of an emergency where a brief incident clip may be shared with emergency contacts. This section details the privacy architecture and introduces The Guard Mode, a specialized operating mode designed for situations where the user needs maximum safety with maximum discretion.

7.1 Zero Cloud Upload Architecture

The codebase includes explicit safeguards to ensure that video frames and audio recordings are processed exclusively in volatile memory. The camera stream is read into byte buffers that are processed by the TFLite interpreter and immediately discarded. No frame data is written to the

device's file system unless the user explicitly enables the optional incident recording feature. Even when incident recording is enabled, the recorded clip is stored in an encrypted container on the device and is only accessible through biometric authentication. The code includes runtime assertions that verify the absence of any network requests containing frame or audio data, and these assertions are validated through automated testing on every commit.

7.2 Black Box Mode

Black Box Mode is a critical feature for users who find themselves in potentially unsafe situations where drawing attention to a safety app could be counterproductive. When activated, the phone's screen goes completely black, showing only a small, translucent "Guardian Shield" icon in the center of the display. Despite the appearance of being powered off, the phone continues to actively monitor the environment through the camera, analyze the audio feed, and run the AI reasoning loop in the background. If an emergency is detected, the system activates the emergency protocol silently, vibrating the phone with a coded pattern and sending alerts to emergency contacts without any visible on-screen activity.

This mode is particularly relevant for scenarios such as walking alone at night, traveling in unfamiliar areas, or situations where the user feels uncomfortable but does not want to escalate by visibly using a safety application. The Black Box Mode leverages Android's foreground service capabilities and iOS's background processing modes to maintain continuous monitoring even when the screen is off. The feature is activated through a configurable trigger, such as pressing the power button three times in quick succession or tapping a discreet widget, making it easy to activate in stressful situations without fumbling through menus.

8. Key GitHub Repositories

The Guard is built on top of several outstanding open-source projects. Each of these repositories represents a critical component of the system and should be bookmarked by any developer working on this project. Understanding the capabilities, limitations, and active development status of each dependency is essential for making informed architectural decisions and troubleshooting integration issues. The following table provides a comprehensive overview of the key repositories, their roles within the system, and the specific versions or branches recommended for The Guard.

Repository	Purpose	Usage in The Guard
------------	---------	--------------------

flutter/flutter	Cross-platform UI framework	Frontend application, camera interface, UI widgets, and platform channels
ultralytics/ultralytics	YOLO object detection models	Person detection and scene analysis via YOLOv8 Nano model
ggerganov/llama.cpp	On-device LLM inference	Running Phi-3 Mini or Llama-3-8B locally for AI reasoning
langchain-ai/langchain	LLM application framework	Structuring the agent's logic, prompt templates, and reasoning chains

Table 5: Key Open-Source Repositories

9. Getting Started: First Steps

The recommended starting point for any developer embarking on The Guard project is to build a simple Flutter application that opens the camera and draws bounding boxes around detected persons in real time. This seemingly simple milestone actually exercises the entire visual pipeline, from camera initialization to model loading to inference to rendering, and provides an excellent foundation for building the more complex features described in the later phases. Once the developer can see a bounding box tracking a person smoothly across the camera feed, they have successfully built "the eyes of The Guard" and can proceed with confidence.

9.1 Step-by-Step First Milestone

Begin by creating a new Flutter project and installing the camera and tflite_flutter packages. Download the YOLOv8 Nano model from the Ultralytics repository and convert it to TFLite format using the export command. Add the converted model file to the project's assets directory and configure the pubspec.yaml to include it. Implement the camera preview widget, the TFLite inference function, and the bounding box overlay painter. Test the application on a physical device (emulators do not have camera access) and verify that person detection works reliably across different lighting conditions and distances. The entire first milestone can typically be completed in 2-4 days by a developer familiar with Flutter and basic machine learning concepts.

9.2 Development Timeline

The complete development of The Guard, from initial concept to production-ready application, is estimated to require approximately three to six months for a solo developer, depending on prior

experience with the relevant technologies. The following timeline provides a realistic breakdown of each phase, including the key deliverables and milestones that mark successful completion of each stage. The timeline assumes part-time development effort of approximately 15-20 hours per week.

Phase	Duration	Key Deliverable
Phase 1: Setup	1 - 2 weeks	Configured Flutter project with camera access and LLM host ready
Phase 2: Vision	2 - 4 weeks	Real-time person detection and fall detection working on device
Phase 3: Agent	3 - 6 weeks	Local LLM reasoning with Guard persona making contextual decisions
Phase 4: Action	2 - 3 weeks	Voice check-in, SMS alerts, and emergency contact system functional
Phase 5: Privacy	2 - 4 weeks	Zero-cloud architecture verified, Black Box Mode implemented

Table 6: Estimated Development Timeline

Building an application that combines computer vision and large language models is at the absolute frontier of AI engineering. The Guard represents not just a product, but an opportunity to master some of the most in-demand skills in the technology industry: on-device machine learning, real-time computer vision, natural language reasoning, and mobile application development. Each phase of development builds on the last, creating a progressively more capable and intelligent system. By the end of this journey, the developer will have built a sophisticated AI application that can genuinely improve people's safety and well-being.